$\mathbb{K}$ definitions as Matching Logic theories, formally

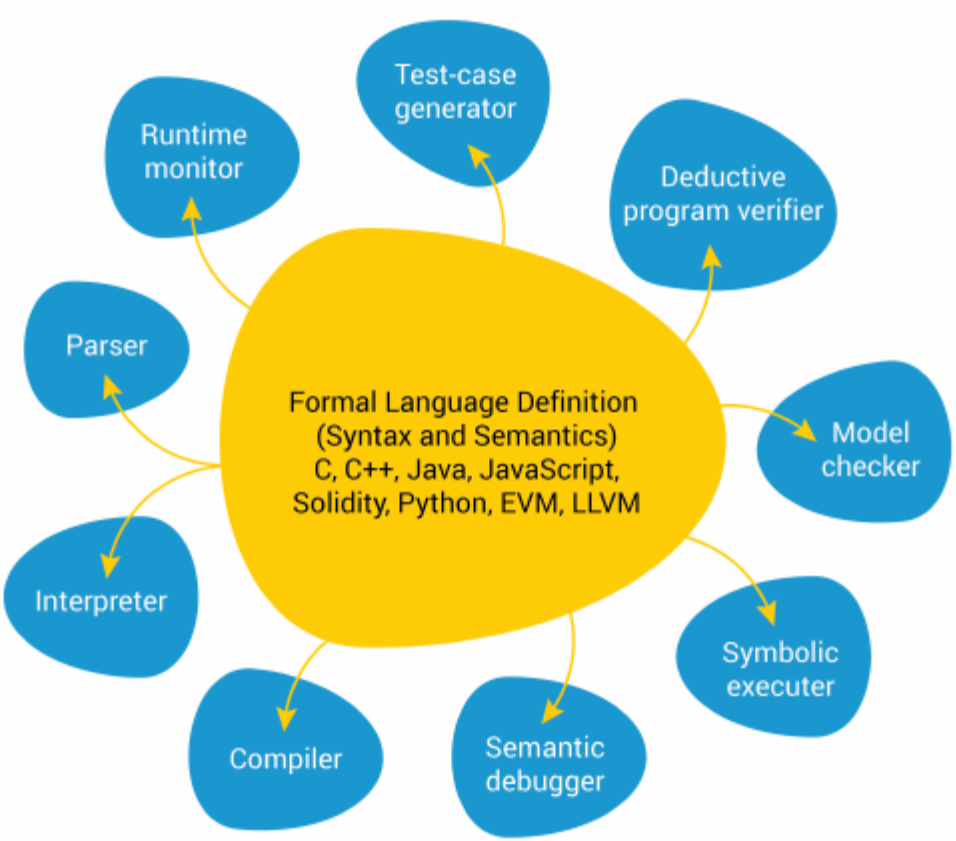
Xiaohong Chen[1], Horaţiu Cheval[2,3], Dorel Lucanu[1,4], Grigore Roşu[1,5]

[1] Pi Squared Inc, [2] University of Bucharest, [3] Institute for Logic and Data Science, [4] Alexandru Ioan Cuza University of Iaşi, [5] University of Illinois at Urbana-Champaign

April 15, 2026,
FoSSaCS, Torino

𝕂 Framework

- Programming languages can be defined using a formal semantics based on rewriting
- Tools can be derived in a generic fashion from the PL definition



𝕂 Framework

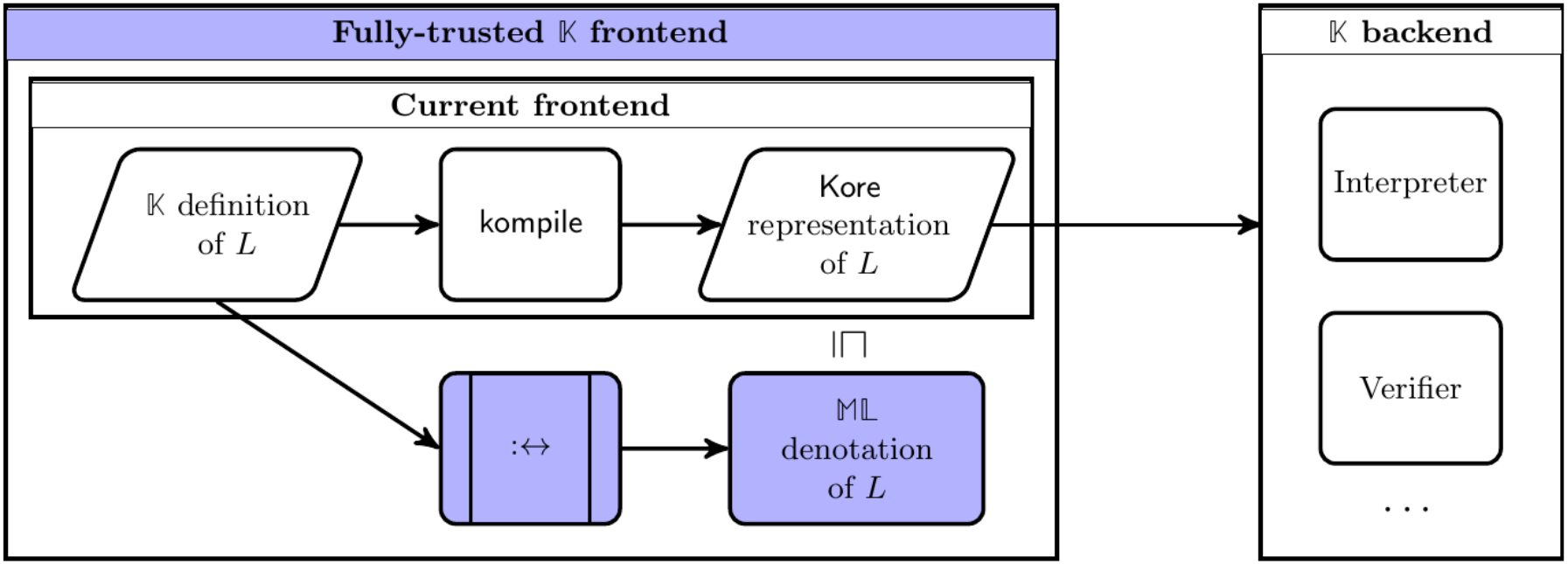
```
module IMP-SYNTAX
  syntax AExp  ::= Int | Id
                 | "-" Int
                 > AExp "+" AExp    [left, seqstrict]
                 ...
  syntax Stmt  ::= Block
                 | Id "=" AExp ";"  [strict(2)]
                 ...
endmodule
module IMP
  imports IMP-SYNTAX
    configuration <T>
                    <k color="green"> $PGM:Pgm </k>
                    <state color="red">
                      .Map
                    </state>
                  </T>

  rule
    <state>... X |-> I ...</state>
    <k> X:Id => I ...</k>
  rule I1 + I2 => I1 +Int I2
  ...
endmodule
```

𝕂 Framework

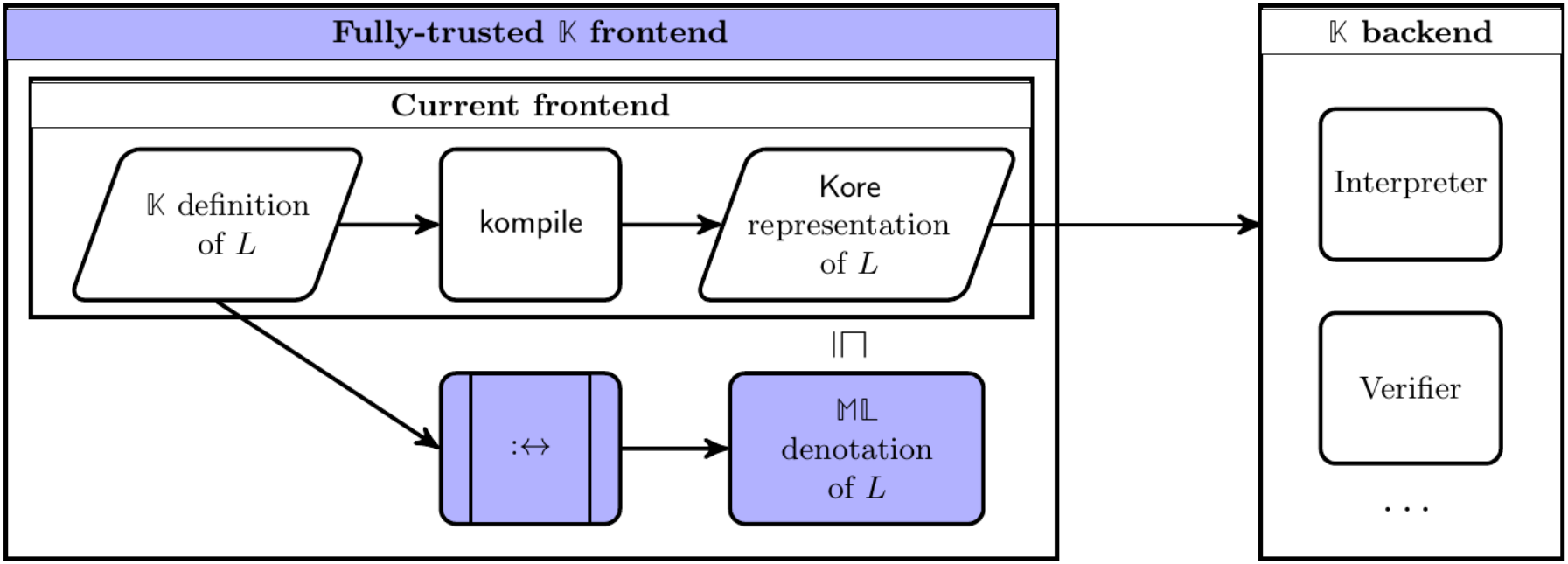
```
int n, sum;
n = 0;
sum = 0;
n = sum + 1;
while ((n <= 100)) {
    sum = sum + n;
    n = n + 1;
}
```

```
> krun sum.imp --definition imp-kompiled --depth 114
<T>
  <k>
    sum = sum + n; ~> n = n + 1; ~> while (n <= 100) {
      sum = sum + n;
      n = n + 1;
    } ~> .K
  </k>
  <state>
    n |-> 4
    sum |-> 6
  </state>
</T>
```

𝕂 Framework

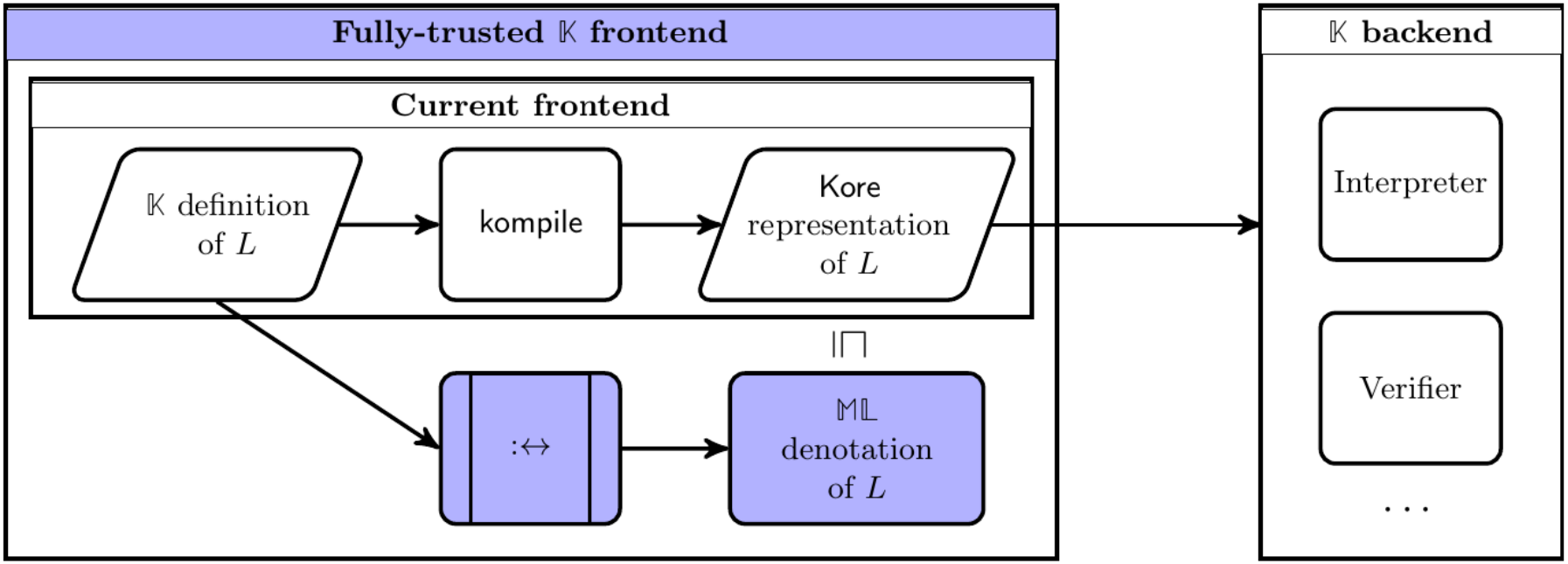

Fully-trusted 𝕂 frontend
Current frontend
𝕂 definition of L
kompile
Kore representation of L
:↔
𝕄𝕃 denotation of L
𝕂 backend
Interpreter
Verifier
...

𝕂 Framework



90 LOC 𝕂 definition → ∼ 8000 LOC Kore theory

𝕂 Framework



90 LOC 𝕂 definition → ∼ 8000 LOC Kore theory

Our contribution:

- A direct denotational semantics of 𝕂 in **Matching Logic**
- A local, composable axiomatization of **abstract rewrite rules**

Matching Logic

- Logical foundation of $\mathbb{K}$; minimal PL-focused system
- Versions: Many sorted ML (Chen, Roşu LMCS '17), Matching μ-Logic (Chen, Roşu LICS '19), Applicative ML (Chen, Roşu '19; Chen, Lucanu, Roşu JLAMP '21)

Matching Logic

- Logical foundation of $\mathbb{K}$; minimal PL-focused system
- Versions: Many sorted ML (Chen, Roşu LMCS '17), Matching μ-Logic (Chen, Roşu LICS '19), Applicative ML (Chen, Roşu '19; Chen, Lucanu, Roşu JLAMP '21)

Patterns, parametrized by a set of symbols Σ:

$$\varphi ::= x \mid X \mid \sigma \in \Sigma \mid \varphi\psi \mid \bot \mid \varphi \to \psi \mid \exists x.\varphi \mid \mu X.\varphi \quad \varphi \text{ positive for } X$$

Matching Logic

- Logical foundation of $\mathbb{K}$; minimal PL-focused system
- Versions: Many sorted ML (Chen, Roşu LMCS '17), Matching μ-Logic (Chen, Roşu LICS '19), Applicative ML (Chen, Roşu '19; Chen, Lucanu, Roşu JLAMP '21)

Patterns, parametrized by a set of symbols Σ:

$$\varphi ::= x \mid X \mid \sigma \in \Sigma \mid \varphi\psi \mid \bot \mid \varphi \to \psi \mid \exists x.\varphi \mid \mu X.\varphi \quad \varphi \text{ positive for } X$$

Interpreted as subsets in a model $(M, (M_\sigma)_{\sigma\in\Sigma}, _\cdot_)$:

- $|\varphi|_\rho \subseteq M$

Matching Logic

- Logical foundation of $\mathbb{K}$; minimal PL-focused system
- Versions: Many sorted ML (Chen, Roşu LMCS '17), Matching μ-Logic (Chen, Roşu LICS '19), Applicative ML (Chen, Roşu '19; Chen, Lucanu, Roşu JLAMP '21)

Patterns, parametrized by a set of symbols Σ:

$$\varphi ::= x \mid X \mid \sigma \in \Sigma \mid \varphi\psi \mid \bot \mid \varphi \to \psi \mid \exists x.\varphi \mid \mu X.\varphi \quad \varphi \text{ positive for } X$$

Interpreted as subsets in a model $(M, (M_\sigma)_{\sigma\in\Sigma}, _\cdot_)$:

- $|\varphi|_\rho \subseteq M$
- Symbol σ denotes $M_\sigma \subseteq M$
- Application $\varphi\psi$ interpreted via $_\cdot_ : M \times M \to 2^M$
- Logical connectives denote set operations; μ is lfp operator

Matching Logic

Some important $\mathbb{ML}$ theories:

Matching Logic

Some important $\mathbb{ML}$ theories:

- **Theory of equality**:
  - Definedness: $\lceil _ \rceil$ with $\forall x. \lceil x \rceil$
  - Derived: $\lfloor \varphi \rfloor \equiv \neg \lceil \neg \varphi \rceil$, $\varphi = \psi \equiv \lfloor \varphi \leftrightarrow \psi \rfloor$, $\varphi \subseteq \psi \equiv \lfloor \varphi \rightarrow \psi \rfloor$, $x \in \varphi \equiv x \subseteq \varphi$

Matching Logic

Some important $\mathbb{ML}$ theories:

- **Theory of equality**:
  - Definedness: $\lceil _ \rceil$ with $\forall x. \lceil x \rceil$
  - Derived: $\lfloor \varphi \rfloor \equiv \neg \lceil \neg \varphi \rceil$, $\varphi = \psi \equiv \lfloor \varphi \leftrightarrow \psi \rfloor$, $\varphi \subseteq \psi \equiv \lfloor \varphi \rightarrow \psi \rfloor$, $x \in \varphi \equiv x \subseteq \varphi$
- **Theory of sorts**:

Matching Logic

Some important $\mathbb{ML}$ theories:

- **Theory of equality**:
  - Definedness: $\lceil _ \rceil$ with $\forall x. \lceil x \rceil$
  - Derived: $\lfloor \varphi \rfloor \equiv \neg \lceil \neg \varphi \rceil$, $\varphi = \psi \equiv \lfloor \varphi \leftrightarrow \psi \rfloor$, $\varphi \subseteq \psi \equiv \lfloor \varphi \rightarrow \psi \rfloor$, $x \in \varphi \equiv x \subseteq \varphi$
- **Theory of sorts**:
  - Sort name constants, inhabitant set $\top_s$; $\varphi{:}s \equiv \varphi \subseteq \top_s$ and $\forall x{:}s.\varphi \equiv \forall x.x \in \top_s \rightarrow \varphi$

Matching Logic

Some important $\mathbb{ML}$ theories:

- **Theory of equality**:
  - Definedness: $\lceil _ \rceil$ with $\forall x. \lceil x \rceil$
  - Derived: $\lfloor \varphi \rfloor \equiv \neg \lceil \neg \varphi \rceil$, $\varphi = \psi \equiv \lfloor \varphi \leftrightarrow \psi \rfloor$, $\varphi \subseteq \psi \equiv \lfloor \varphi \rightarrow \psi \rfloor$, $x \in \varphi \equiv x \subseteq \varphi$
- **Theory of sorts**:
  - Sort name constants, inhabitant set $\top_s$; $\varphi{:}s \equiv \varphi \subseteq \top_s$ and $\forall x{:}s.\varphi \equiv \forall x.x \in \top_s \rightarrow \varphi$
  - Product sorts $s_1 \otimes s_2$, power sorts 2^s, function sorts $[s_1 \rightarrow s_2]$

Matching Logic

Some important $\mathbb{ML}$ theories:

- **Theory of equality**:
  - Definedness: $\lceil _ \rceil$ with $\forall x. \lceil x \rceil$
  - Derived: $\lfloor \varphi \rfloor \equiv \neg \lceil \neg \varphi \rceil$, $\varphi = \psi \equiv \lfloor \varphi \leftrightarrow \psi \rfloor$, $\varphi \subseteq \psi \equiv \lfloor \varphi \rightarrow \psi \rfloor$, $x \in \varphi \equiv x \subseteq \varphi$
- **Theory of sorts**:
  - Sort name constants, inhabitant set $\top_s$; $\varphi{:}s \equiv \varphi \subseteq \top_s$ and $\forall x{:}s.\varphi \equiv \forall x.x \in \top_s \rightarrow \varphi$
  - Product sorts $s_1 \otimes s_2$, power sorts 2^s, function sorts $[s_1 \rightarrow s_2]$
  - Binders $\lambda x.\varphi$ encodable via power sorts (Chen, Roşu ICFP '20)

Contexts

Contexts

- Intuitively: patterns $\varphi(\square)$ with a 'hole' $\square$

Contexts

- Intuitively: patterns $\varphi(\square)$ with a 'hole' $\square$
- Straightforward by extending the syntax, but that is no longer $\mathbb{ML}$

Contexts

- Intuitively: patterns $\varphi(\Box)$ with a 'hole' $\Box$
- Straightforward by extending the syntax, but that is no longer $\mathbb{ML}$
- **Idea:** reify contexts as patterns of the form $\gamma\Box{:}s.\varphi$:

$$\begin{aligned} \gamma\Box{:}s.\varphi &:\equiv \mathsf{gamma}(\mathsf{intension}(\exists\Box.\,\langle\Box,\varphi\rangle)), \\ C[x] &:\equiv \mathsf{plug}(C,x) \end{aligned}$$

and axiomatize the operations

Contexts

- Intuitively: patterns $\varphi(\Box)$ with a 'hole' $\Box$
- Straightforward by extending the syntax, but that is no longer $\mathbb{ML}$
- **Idea:** reify contexts as patterns of the form $\gamma\Box{:}s.\varphi$:

$$\begin{aligned} \gamma\Box{:}s.\varphi &:\equiv \mathsf{gamma}(\mathsf{intension}(\exists\Box.\,\langle\Box,\varphi\rangle)), \\ C[x] &:\equiv \mathsf{plug}(C,x) \end{aligned}$$

  and axiomatize the operations
- β-reduction for plugging can be derived:

$$(\gamma\Box{:}s.\varphi)[\psi] = \varphi[\psi/\Box]$$

- Composition $C_2 \circ C_1$: function composition on contexts

Configurations

```
> krun sum.imp --definition imp-kompiled --depth 114
<T>
  <k>
    sum = sum + n; ~> n = n + 1; ~> while (n <= 100) {
      sum = sum + n;
      n = n + 1;
    } ~> .K
  </k>
  <state>
    n |-> 4
    sum |-> 6
  </state>
</T>
```

Configurations

```
> krun sum.imp --definition imp-kompiled --depth 114
<T>
  <k>
    sum = sum + n; ~> n = n + 1; ~> while (n <= 100) {
      sum = sum + n;
      n = n + 1;
    } ~> .K
  </k>
  <state>
    n |-> 4
    sum |-> 6
  </state>
</T>
```

- Tree-like terms
- k-cell: associative list of computations

Configurations

```
> krun sum.imp --definition imp-kompiled --depth 114
<T>
  <k>
    sum = sum + n; ~> n = n + 1; ~> while (n <= 100) {
      sum = sum + n;
      n = n + 1;
    } ~> .K
  </k>
  <state>
    n |-> 4
    sum |-> 6
  </state>
</T>
```

- Tree-like terms
- k-cell: associative list of computations

$$
\begin{aligned}
&\langle\!|\mathsf{T}|\!\rangle(\\
&\quad \langle\!|\mathsf{k}|\!\rangle(\mathsf{assign}(\mathit{sum}, \mathsf{plus}(\mathit{sum}, n)) \curvearrowright \mathsf{assign}(n, \mathsf{plus}(n, 1)) \curvearrowright w \curvearrowright \mathsf{dotK}),\\
&\quad \langle\!|\mathsf{state}|\!\rangle(n \mapsto 4 \ \mathsf{mapConcat}\ \mathit{sum} \mapsto 6)\\
&)
\end{aligned}
$$

Rule axiomatization: kompile

`kompile` output (roughly) for `IMP`:

$$\langle\!|\mathsf{T}|\!\rangle(\langle\!|\mathsf{k}|\!\rangle(\mathsf{plus}(i_1, i_2) \curvearrowright \kappa), s{:}\mathsf{Cell}\langle\mathsf{state}\rangle)$$

$$\Rightarrow$$

$$\langle\!|\mathsf{T}|\!\rangle(\langle\!|\mathsf{k}|\!\rangle(i_1 +_{\mathsf{Int}} i_2 \curvearrowright \kappa), s{:}\mathsf{Cell}\langle\mathsf{state}\rangle)$$

Rewriting: symbol $\bullet$ (**next**); $\varphi \Rightarrow \psi :\equiv \varphi \to \bullet\psi$.

Rule axiomatization: kompile

```
configuration
<T color="yellow">
    <threads>
    <thread multiplicity="*" type="Map" initial="">
        <id> 0 </id>
        <k> $PGM:Pgm </k>
    </thread>
    </threads>
    <state> .Map </state>
    <next-id> 1 </next-id>
</T>
```

Rule axiomatization: kompile

```
configuration
<T color="yellow">
    <threads>
    <thread multiplicity="*" type="Map" initial="">
        <id> 0 </id>
        <k> $PGM:Pgm </k>
    </thread>
    </threads>
    <state> .Map </state>
    <next-id> 1 </next-id>
</T>
```

`kompile` output (roughly):

$$\langle\!|\mathsf{T}|\!\rangle(\langle\!|\mathsf{threads}|\!\rangle(\mathit{tid} \mapsto \langle\!|\mathsf{thread}|\!\rangle(\mathit{tid}, \langle\!|\mathsf{k}|\!\rangle(\mathsf{plus}(i_1, i_2) \curvearrowright \kappa))\ \mathsf{mapConcat}\ t),$$
$$s{:}\mathsf{Cell}\langle\mathsf{state}\rangle,\ n{:}\mathsf{Cell}\langle\mathsf{next\text{-}id}\rangle)$$

$$\Rightarrow$$

$$\langle\!|\mathsf{T}|\!\rangle(\langle\!|\mathsf{threads}|\!\rangle(\mathit{tid} \mapsto \langle\!|\mathsf{thread}|\!\rangle(\mathit{tid}, \langle\!|\mathsf{k}|\!\rangle(i_1 +_{\mathsf{Int}} i_2 \curvearrowright \kappa))\ \mathsf{mapConcat}\ t),$$
$$s{:}\mathsf{Cell}\langle\mathsf{state}\rangle,\ n{:}\mathsf{Cell}\langle\mathsf{next\text{-}id}\rangle)$$

Rule axiomatization: with contexts

$\mathbb{K}$ rule:

```
rule I1 + I2 => I1 +Int I2
```

Rule axiomatization: with contexts

$\mathbb{K}$ rule:

```
rule I1 + I2 => I1 +Int I2
```

$\mathbb{ML}$ axiom:

$$\forall C\colon \mathsf{Context}^{\mathsf{Cell}\langle\mathsf{T}\rangle}_{\mathsf{Cell}\langle\mathsf{k}\rangle}.\forall i_1, i_2\colon \mathsf{Int}.\forall \kappa\colon \mathsf{K}.$$
$$(C \circ C_\kappa)[\mathsf{plus}(i_1, i_2)] \Rightarrow (C \circ C_\kappa)[i_1 +_{\mathsf{Int}} i_2],$$

where $C_\kappa :\equiv \gamma\Box\colon \mathsf{K}.\langle\!|\mathsf{k}|\!\rangle(\Box \curvearrowright \kappa)$.

Rule axiomatization: with contexts

$\mathbb{K}$ rule:

```
rule I1 + I2 => I1 +Int I2
```

$\mathbb{ML}$ axiom:

$$\forall C\colon \mathsf{Context}^{\mathsf{Cell}\langle\mathsf{T}\rangle}_{\mathsf{Cell}\langle\mathsf{k}\rangle}.\forall i_1, i_2\colon \mathsf{Int}.\forall \kappa\colon \mathsf{K}.$$
$$(C \circ C_\kappa)[\mathsf{plus}(i_1, i_2)] \Rightarrow (C \circ C_\kappa)[i_1 +_{\mathsf{Int}} i_2],$$

where $C_\kappa :\equiv \gamma\Box\colon \mathsf{K}.\langle\!|\mathsf{k}|\!\rangle(\Box \curvearrowright \kappa)$.

Importantly, the same axiom can be used in any configuration.

Strictness: heating and cooling

$\mathbb{K}$ attribute:

```
syntax Stmt ::= Id "=" AExp ";" [strict(2)]
```

Strictness: heating and cooling

$\mathbb{K}$ attribute:

```
syntax Stmt ::= Id "=" AExp ";" [strict(2)]
```

$\mathbb{ML}$ axioms:

$$\forall C\colon \mathsf{Context}^{\mathsf{Cell}\langle\mathsf{T}\rangle}_{\mathsf{Cell}\langle\mathsf{k}\rangle}.\forall \kappa\colon \mathsf{K}.\forall i\colon \mathsf{Id}.\forall e\colon \mathsf{AExp}.$$
$$(C \circ C_\kappa)[\mathsf{assign}(i, e)] \wedge \neg\,\mathsf{KResult}(e) \Rightarrow (C \circ C_\kappa)[e \curvearrowright C_{\mathsf{assign},2}]$$
$$\forall C\colon \mathsf{Context}^{\mathsf{Cell}\langle\mathsf{T}\rangle}_{\mathsf{Cell}\langle\mathsf{k}\rangle}.\forall \kappa\colon \mathsf{K}.\forall i\colon \mathsf{Id}.\forall e\colon \mathsf{AExp}.$$
$$(C \circ C_\kappa)[e \curvearrowright C_{\mathsf{assign},2}] \wedge \mathsf{KResult}(e) \Rightarrow (C \circ C_\kappa)[C_{\mathsf{assign},2}[e]]$$

where

$C_\kappa \equiv \gamma\Box\colon \mathsf{K}.\langle\!|\mathsf{k}|\!\rangle(\Box \curvearrowright \kappa)$

$C_{\mathsf{assign},2} \equiv \gamma\Box\colon \mathsf{AExp}.\,\mathsf{assign}(i, \Box)$

Conclusion

- A direct denotational semantics of $\mathbb{K}$ in $\mathbb{ML}$

Conclusion

- A direct denotational semantics of $\mathbb{K}$ in $\mathbb{ML}$
- Other $\mathbb{K}$ features: syntax productions (as inductive sorts), static and dynamic attributes, builtin datatypes

Conclusion

- A direct denotational semantics of $\mathbb{K}$ in $\mathbb{ML}$
- Other $\mathbb{K}$ features: syntax productions (as inductive sorts), static and dynamic attributes, builtin datatypes
- Uniform treatment of abstract rules, dynamic attributes via contexts

Future work:

- Formalizing the theory and implementing the translation

Conclusion

- A direct denotational semantics of $\mathbb{K}$ in $\mathbb{ML}$
- Other $\mathbb{K}$ features: syntax productions (as inductive sorts), static and dynamic attributes, builtin datatypes
- Uniform treatment of abstract rules, dynamic attributes via contexts

Future work:

- Formalizing the theory and implementing the translation
- Verifying `kompile`'s Kore output is a refinement of $L^{\mathbb{ML}}$

Conclusion

- A direct denotational semantics of $\mathbb{K}$ in $\mathbb{ML}$
- Other $\mathbb{K}$ features: syntax productions (as inductive sorts), static and dynamic attributes, builtin datatypes
- Uniform treatment of abstract rules, dynamic attributes via contexts

Future work:

- Formalizing the theory and implementing the translation
- Verifying `kompile`'s Kore output is a refinement of $L^{\mathbb{ML}}$